Group Proposal: Astrophysics Literature Multi-Label Classification Based on AstroConcepts

1. Problem Selection and Motivation

Selected Problem: Multi-label keyword (UAT concept) prediction for astrophysics literature abstracts. Given a paper's title and abstract, the model automatically predicts the Unified Astronomy Thesaurus (UAT) concepts covered by the paper (such as "Galaxy evolution", "Exoplanet atmospheres", etc.).

Why Choose This Problem: First, it naturally aligns with the course's five key concepts. Multi-label text classification can simultaneously cover classical models (Naive Bayes, Logistic Regression), recurrent networks (LSTM/GRU), pretrained Transformers (BERT/astroBERT), Autoencoders for unsupervised visualization, and LIME/SHAP for interpretability analysis. Second, the dataset already exists publicly and has been baseline-tested in academic papers, requiring no additional time for data collection. Third, the dataset exhibits severe long-tail label distribution (76% of labels have fewer than 50 training samples), which itself constitutes a meaningful machine learning challenge—the group must address class imbalance strategies in the report, rather than simply pursuing overall accuracy.

2. Dataset

Dataset: adsabs/SciX_UAT_keywords (Hugging Face).

Scale: 21,702 astrophysics paper abstracts (title + abstract), with 2,367 UAT concepts as the label space, and 1,864 concepts actually appearing. Each abstract corresponds to 4.31 labels on average.

Label Distribution: Severe long-tail distribution. 76.1% of labels have fewer than 50 samples (Tail), 23% are in the 50–500 range (Torso), and only 17 labels have more than 500 samples (Head).

Split: Following the original paper's protocol, using label-aware stratified splitting, with approximately 85% training / 15% testing. Group A is responsible for completing the split in the first week and saving it as a fixed file to ensure all members use the same data partition.

3. NLP Methods and Customization

Course Concept	Implementation in This Project	Whether Customization Is Required
Rule-based Models	TF-IDF + Multinomial Naive Bayes, Logistic Regression (one-vs-rest)	No, use scikit-learn directly
Recurrent Network	LSTM / GRU, using pretrained GloVe embeddings or training from scratch	Requires customization of multi-label output layer and BCEWithLogitsLoss
Pretrained Transformer	astroBERT fine-tuning + multi-label classification head	Requires customization of multi-label classification head (num_labels=2367)

Autoencoder	Text Autoencoder, compressing TF-IDF or BERT embeddings to 2D for visualization	Requires customization of encoder/decoder structure
Interpretability	LIME and SHAP applied to the best-performing model	No, use lime and shap libraries directly

Key References: astroBERT is a domain-pretrained model trained by ADS on astronomy text. The AstroConcepts paper's baseline results show that astroBERT achieves the highest Macro-F1 among supervised models (0.324), providing Group C with a clear comparison target.

4. Packages and Tools

PyTorch + Hugging Face Transformers: For training astroBERT and LSTM/GRU. PyTorch's flexibility facilitates customizing multi-label loss functions (BCEWithLogitsLoss + class weights).

scikit-learn: For TF-IDF vectorization, Naive Bayes, Logistic Regression, and calculation of evaluation metrics.

LIME (lime) and SHAP (shap): For interpretability analysis.

Google Cloud Deep Learning VM: Pre-installed with PyTorch, CUDA, scikit-learn, and JupyterLab, can be launched directly with GPU instances. This is the key to completing the project within the $50 GCP credit—the group only needs to activate the instance when training is needed, and shut it down immediately after use.

5. NLP Tasks

Core Task: Multi-label text classification. Input: title + abstract. Output: subset of 2,367 UAT concepts (typically 1–12).

Auxiliary Tasks: Autoencoder latent space 2D visualization (color-coded by the most frequent UAT category of each abstract, observing separability); LIME/SHAP token-level attribution analysis (showing which words in the abstract drive or suppress specific UAT predictions).

6. Evaluation Metrics

Using Macro-F1 as the primary metric (due to long-tail distribution, micro-F1 is dominated by head classes). Simultaneously reporting Precision@1/3/5 and Recall@1/3/5 to evaluate ranking quality. Group D is responsible for frequency-stratified evaluation: reporting F1 separately for Head (>500), Torso (50–500), and Tail (<50) labels, revealing whether the model truly learns rare concepts.

7. Computing Resource Plan (GCP $50)

Using Google Cloud Deep Learning VM images, selecting GPU instances (such as NVIDIA T4), pre-installed with PyTorch and CUDA. The $50 credit is sufficient to cover: astroBERT fine-tuning (approximately 2–4 hours), LSTM/GRU training (approximately 1 hour), Autoencoder and interpretability analysis (approximately 2 hours). Key measure: instances are activated only during training and immediately shut down after completion, avoiding idle resource consumption. Group A is responsible for creating a shared GCP project and managing credit usage.

8. Group Division of Labor and Timeline

Week	Task	Responsible
1	Create GitHub repo; download dataset; complete train/test split; write utils.py (data loading + evaluate_model)	A
1–2	EDA (label distribution log-log plot, text length distribution); TF-IDF + NB/LR baseline	A
2–3	LSTM/GRU training and tuning (hidden size, layers, dropout)	B
3–4	astroBERT fine-tuning (learning rate {2e-5, 3e-5, 5e-5}, epochs 2–8)	C
4–5	Text Autoencoder training + 2D scatter plot; LIME/SHAP analysis	D
5–6	Consolidate all results, write final report, create presentation slides	All members
6	Rehearse presentation (12–15 minutes)	All members

9. GitHub Repository Structure

```
FinalProject-GroupX/
├── README.md
├── Group-Proposal/proposal.pdf
├── Final-Group-Project-Report/report.pdf
├── Final-Group-Presentation/slides.pdf
└── Code/
    ├── README.md          # Running order: 01 → 02 → 03 → 04 → 05 → 06
    ├── utils.py           # Data loading + evaluate_model()
    ├── 01_split_data.py
    ├── 02_baseline_nb_lr.py
    ├── 03_rnn_lstm_gru.py
    ├── 04_bert_finetune.py
    ├── 05_autoencoder_viz.py
    └── 06_lime_shap.py
```

Key utils.py Interface (A completes in the first week for use by B/C/D):

```
def load_data(split="train"):
    """Return: texts (list[str]), labels (multi-hot np.array)"""
    ...

def evaluate_model(y_true, y_pred, y_score=None):
    """Return dict: accuracy, macro_f1, micro_f1, precision, recall,
       precision@1, precision@3, precision@5, recall@1, recall@3, recall@5"""
    ...
```

10. Risks and Alternatives

Risk 1: astroBERT multi-label head has many parameters (2367 labels) and unstable training. Mitigation: First run through the pipeline with BERT-base, then switch to astroBERT; use BCEWithLogitsLoss with class weights (inverse frequency) to address imbalance.

Risk 2: $50 GCP credit insufficient. Mitigation: Strictly implement “shut down immediately after use”; Group C prioritizes using smaller batch_size (8 or 4) + gradient accumulation; if the credit is still tight, Group A can run CPU-only tasks (TF-IDF baseline, EDA) on their local machine to save cloud resources.

Risk 3: Autoencoder latent space visualization not ideal. Mitigation: Try two inputs: TF-IDF and astroBERT embeddings. If the 2D scatter plot does not show clustering, the group can discuss in the report “the difficulty of unsupervised representation learning in long-tail multi-label scenarios”—this is itself a valuable negative result.

References

Alkan, A. K., et al. (2026). AstroConcepts: A Large-Scale Multi-Label Classification Corpus for Astrophysics. arXiv:2604.02156.

ADS KAILAS model and dataset. Harvard ADS User Group 2025.

Hugging Face Dataset: adsabs/SciX_UAT_keywords.

Google Cloud Deep Learning VM Documentation.

Shukla, M. A. (2025). Interpreting BERT Using LIME and SHAP. TechRxiv.